EaglePretest

Code Analysis Documentation

Backend (pre-test engine) | GUI (Tkinter front-end) | How they connect

Item	Value
Software	EaglePretest v1.0.0 (SOFTWARE_VERSION in constants.py)
Supported modules	QCX216 (LTE), MDM9X07 (LTE), SDX35 (NR)
Language / runtime	Python 3, Tkinter, pyserial, openpyxl, Pillow, ReportLab; packaged with PyInstaller
Author of the tool	Anas Akasyah Bin Abdul Patas
Code base analysed	EaglePretest_Development/src (7 .py files, 5,616 lines) and tests/ (10 files)
Document date	25 October 2026

Contents

Contents 2

1. Overview 3

1.1 Repository layout . . . 3

1.2 Third-party dependencies . . . 3

2. Backend Analysis 4

2.1 Data model - test_case.py . . . 4

2.2 Configuration - constants.py . . . 4

2.3 Execution pipeline (per module) . . . 5

2.4 Response grading - at_execute_command() . . . 5

2.5 Module differences . . . 6

2.6 How the backend files connect . . . 7

2.7 Worked examples . . . 11

2.8 Backend observations . . . 13

3. GUI Analysis 14

3.1 Structure of gui.py . . . 14

3.2 Start-up and login . . . 14

3.3 Main window . . . 14

3.4 Pre-check model . . . 15

3.5 Online mode (JIRA) . . . 15

3.6 Run control and output parsing . . . 15

3.7 Results and housekeeping . . . 16

3.8 Threading model . . . 16

3.9 GUI observations . . . 16

4. Connection between GUI and Backend 17

4.1 Integration points . . . 18

4.2 Process launch contract . . . 18

4.3 Stdout protocol . . . 18

4.4 End-to-end sequence of a run . . . 19

4.5 Shared file-system contract . . . 19

4.6 Rules defined twice . . . 20

4.7 Assessment of the coupling . . . 20

5. Prioritised Recommendations 20

Appendix A - Test suite . . . 21

1. Overview

EaglePretest automates the AT-command pre-test that the software team runs on every firmware build before release. The tool reads the build parameters of a Jenkins build (from an Excel sheet), derives the expected modem responses, sends 44 AT commands to the modem through its USB AT port, and grades each response as PASS, FAIL or UNSUPPORTED (shown as SKIP). Results are written to a timestamped log, shown live in the GUI, exported as a PDF report and, in online mode, posted back to the JIRA ticket.

The code base is split into two layers that this document analyses separately:

- **Backend** - the original console pre-test engine: `constants.py`, `test_case.py`, `main.py` and one script per module (`qcx216.py`, `mdm9x07.py`, `sdx35.py`).
- **GUI** - `gui.py`, a single 3,882-line Tkinter application that wraps the backend without modifying it, plus JIRA integration, reporting and packaging.
- **Connection** - the GUI runs each pre-test in a child process (the same executable started with `--worker`) and reads the backend's console output as a line-based protocol. Section 4 documents that contract.

1.1 Repository layout

Path	Lines	Layer	Role
`src/constants.py`	46	Backend	Version, serial settings, reset delays, SIM lengths, branch names
`src/test_case.py`	439	Backend	Data model: TestCaseIndex, ATCommonInfo, ATTestCase and the 44-command list
`src/main.py`	120	Backend	Legacy console entry; read_jenkins_value() (reused by the GUI)
`src/qcx216.py`	383	Backend	QCX216 pre-test pipeline
`src/mdm9x07.py`	374	Backend	MDM9X07 pre-test pipeline (JIRA project SW9X07)
`src/sdx35.py`	372	Backend	SDX35 (NR) pre-test pipeline with skip rules
`src/gui.py`	3,882	GUI	Login, main window, worker entry, JIRA, PDF report
`src/EaglePretest.spec`	-	Build	PyInstaller one-file build of gui.py; icon cut from the logo
`tests/*.py`	1,142	Tests	GUI tests against a mock JIRA and a fake worker
`docs/html`	-	Docs	Doxygen output generated from the ## @brief comments

1.2 Third-party dependencies

Library	Used by	Purpose
pyserial	Backend, GUI	Serial I/O to the modem; port enumeration (serial.tools.list_ports)
openpyxl	Backend	Read the Jenkins Build Parameter sheet (.xlsx)
tkinter / ttk	GUI	Windows, widgets, Treeview table, dialogs
Pillow (PIL)	GUI, build	Logo emblem extraction for window icon, .ico and report logo
reportlab	GUI	PDF result report (lazy import inside write_report_pdf)
urllib / json / base64	GUI	JIRA REST calls with Basic auth (no requests dependency)
ctypes (advapi32, shcore)	GUI	Windows Credential Manager token store, DPI awareness, work area
PyInstaller	Build	Single EaglePretest.exe, windowed (console=False)

2. Backend Analysis

The backend is the pre-test engine. It has no knowledge of the GUI: every module script exposes one entry function, prints its progress to stdout and writes the same text to a log file. All configuration lives in module-level objects that the scripts mutate before the run.

2.1 Data model - test_case.py

Element	Kind	Description
`ATTestCase`	@dataclass	test_case_id, at_cmd_index, at_command, expected_response, response_line=1, prefix_value=None, exact_match=False, has_prefix=False
`at_cmd_test_case_list`	list[ATTestCase]	The 44 commands, in execution order. Expected values are placeholders that are overwritten at runtime from the Excel sheet and SIM inputs.
`ATCommonInfo`	@dataclass	Build information: branch_name, sub_version, svn, version, version_info, sim_ccid/imsi/imei, oem, version_qgmr, certversion, platform_name, modemapp, linuxapp
`at_cmd_general_info_list`	list[ATCommonInfo]	Single shared record (index 0) with sample defaults; the module scripts refill it from Excel
`TestCaseIndex`	IntEnum	Position of each command in the list (CD_01_000_1 = 0 ... CD_05_004_1 = 43). Used to patch expected responses by name instead of by number.

Test case groups:

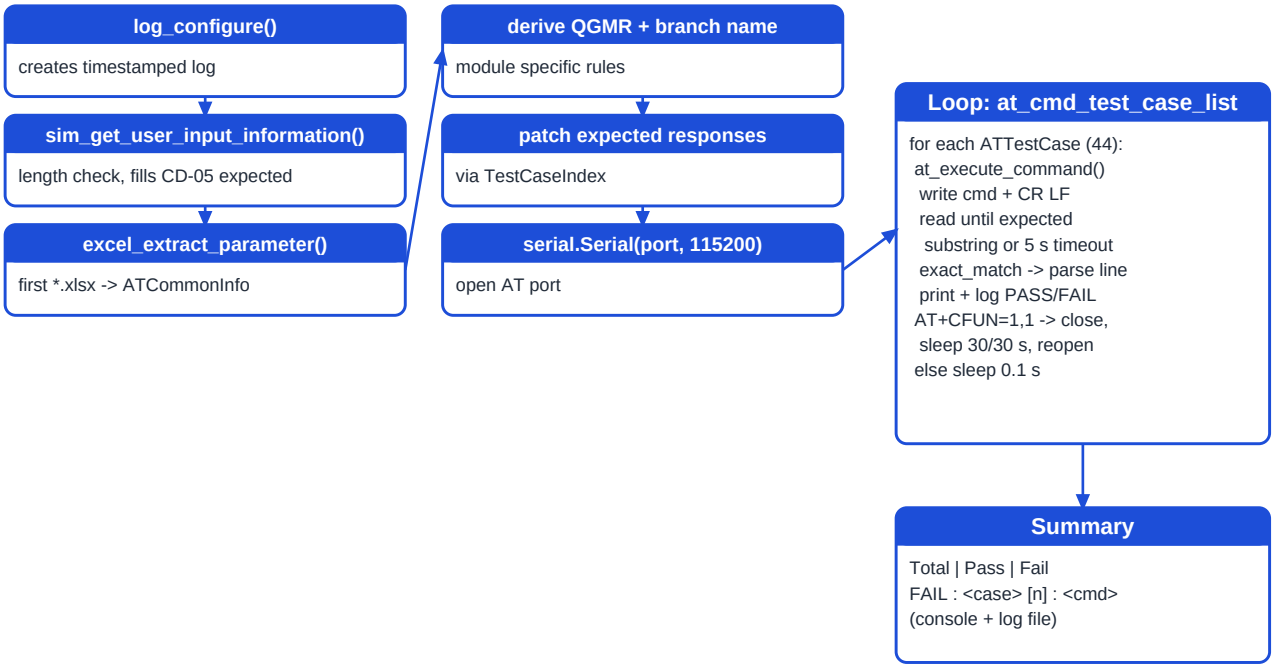
Group	Cmds	Commands	Checks
CD-01-000	2	AT, ATE1	Modem alive; echo on (needed by the response parser)
CD-01-001	2	AT+GMI, AT+CGMI	Manufacturer = branch name (Eagle Wireless / NetPrisma / Quectel), exact
CD-01-002	2	AT+GMM, AT+CGMM	Model query answers OK
CD-01-003	6	AT+GMR, AT+CGMR, ATI, AT+CVERSION, AT+QGMR, AT+CSUB	Version strings; ATI/CVERSION strip a prefix; QGMR exact match to derived version
CD-02-001	1	AT+EGMR=0,9	SVN from Excel
CD-04-001/2	3	AT+QSVN=?, AT+QSVN?, AT+QSVN=99	QSVN syntax, read, invalid value
CD-04-003	22	AT+QSVN=..., AT+CFUN=1,1, AT+EGMR=0,9, AT+QNVFD	Set SVN, reset modem (4 resets), verify persistence; restore
CD-05-001..004	6	AT+QCCID, AT+GSN, AT+CGSN, AT+QGSN, AT+CIMI, AT+EGMR=?	SIM CCID / IMEI / IMSI from user input

2.2 Configuration - constants.py

Constant	Value	Use
`SOFTWARE_VERSION`	1.0.0	Single source of the version (startup banner, GUI title, report)
`BAUD_RATE / SEND_TIMEOUT`	115200 / 5 s	Serial speed and per-command response timeout
`COMMAND_DELAY_SEC`	0.1 s	Poll interval and pause between commands
`RESET_AT_COMMAND`	AT+CFUN=1,1	Triggers the close / wait / reopen sequence
`RESET_DELAY_*_SEC`	QCX216 30, MDM9X07 30, SDX35 90	Wait after reset (SDX35 value is unused: no reset handling)
`SIM_*_LEN`	CCID 20, IMEI 15, IMSI 15	Input validation (backend and GUI)
`BRANCH_NAME_*`	Eagle Wireless / NetPrisma / Quectel	Expected manufacturer string
`AT_CMD_MIN_LINE_TO_PARSE`	2	Minimum response lines before exact-match parsing

2.3 Execution pipeline (per module)

Each module script defines the same six functions. `pretest_<module>()` is the only public entry point; it is called by main.py (console) or by the GUI worker.



Every module script (qcx216.py, mdm9x07.py, sdx35.py) follows this same pipeline; only steps 3-5 and the reset/skip rules differ.

Figure 1 - Backend pipeline of pretest_<module>()

Function	Responsibility
`log_configure()`	logging.basicConfig to Eagle_Wireless_SW_AT_Pretest_Log_<timestamp>.txt next to the script / exe. Returns the path.
`log_write(text)`	logging.info wrapper.
`sim_get_user_input_information(ccid, imei, imsi)`	Checks lengths only; stores the SIM values and writes them into the CD-05 expected responses.
`read_jenkins_value(path, name)`	Opens the workbook, finds the 'Jenkins Parameter' and 'Value' header columns in row 1, returns the value as str, or the string "None" when the parameter is missing.
`excel_extract_parameter()`	Picks the first *.xlsx in the app folder, fills ATCommonInfo, derives the QGMR version and branch name, and patches expected responses through TestCaseIndex.
`at_execute_command(port, tc, i, list, timeout)`	Writes the command, reads until the expected text appears or the timeout expires, optionally parses one exact line, prints and logs PASS/FAIL.
`pretest_<module>(user, port, ccid, imei, imsi)`	Orchestrates the run, counts results, handles AT+CFUN=1,1 resets and prints the Test Summary and FAIL list.

2.4 Response grading - at_execute_command()

```
write(cmd + CRLF)
loop until 5 s:
    buffer += read(in_waiting)
    if expected_response in buffer: received = True; break   # substring match
    sleep(0.1)
if received and exact_match:
    lines = non-empty stripped lines of buffer
    actual = lines[response_line]            # line 0 is the echoed command (ATE1)
    if has_prefix: actual = actual.removeprefix(prefix_value).strip()
    PASS only if actual == expected_response
elif received: PASS
else: FAIL
```

Two grading modes exist: a **substring** check (default) and an **exact line** check for manufacturer and version commands. The exact check relies on ATE1 echo so that line 0 is the command and line 1 (or line 3 for ATI) is the value.

2.5 Module differences

The three module scripts are about 95 % identical (verified with diff). The differences are:

Aspect	QCX216	MDM9X07	SDX35
Branch key in Excel	branch_name	branch_name	branchname
OEM key	OEM	oem_id	(not read)
QGMR rule	version [_OEM] [_modemapp.modemapp]; OEM skipped if already in version	certversion _modemapp .linuxapp[0:2], else certversion	certversion _modemapp .linuxapp[0:2], else certversion
GMR/CGMR/ATI/CVERSION expect	certversion (all 4)	certversion; ATI expects version	version (all 4)
AT+QSVN=99 expects	OK	ERROR	skipped
CD-04 group, AT+QGSN	run	run	UNSUPPORTED (counted as pass)
Reset handling	close, sleep 30 s, reopen	close, sleep 30 s, reopen	none (0.1 s delay)
CD-04-003 #13/#20	A(\d{2}) from version	A(\d{2}) from version	A(\d{2}) from version

2.6 How the backend files connect

The backend has three layers: **main.py** picks a module, each **module script** runs the test, and the two shared files, **test_case.py** (data) and **constants.py** (settings), are imported by all three module scripts. The module scripts do not import each other or main.py, and main.py does not import constants.py.

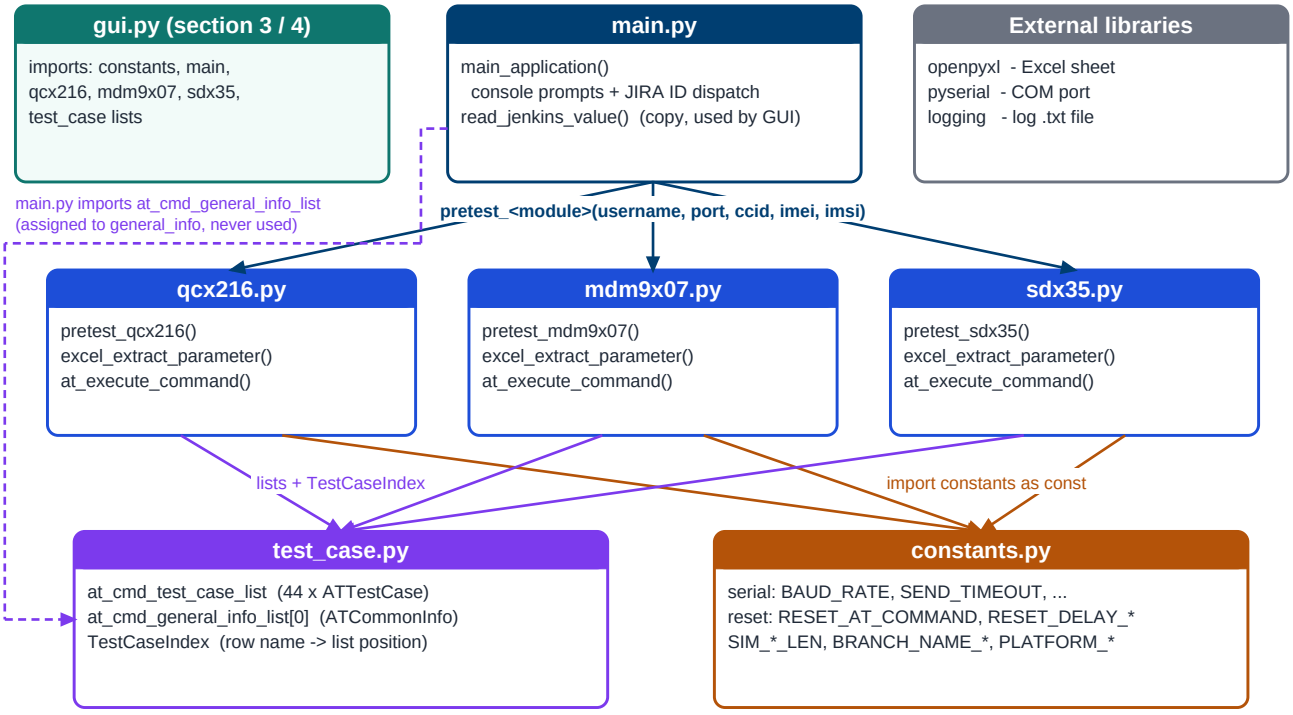


Figure 2 - Import dependencies between the backend files (arrow = imports / calls)

From	To	What is imported / called	How it is used
`main.py`	`qcx216, mdm9x07, sdx35`	`pretest_qcx216(), pretest_mdm9x07(), pretest_sdx35()`	Called once after the JIRA ID prompt
`main.py`	`test_case.py`	`at_cmd_general_info_list`	Assigned to general_info but never used
`module script`	`test_case.py`	`at_cmd_test_case_list`	Iterated by the run loop; expected_response overwritten
`module script`	`test_case.py`	`at_cmd_general_info_list[0]`	Filled from the SIM input and the Excel sheet
`module script`	`test_case.py`	`TestCaseIndex`	Names the 18 rows whose expected value is patched
`module script`	`constants.py`	`import constants as const`	Settings in the table below
`gui.py`	`all of the above`	`see section 4.1`	Library use and worker process

Constant usage per file (number = references in the file)

Constant	main	qcx216	mdm9x07	sdx35	gui
SOFTWARE_VERSION		via banner	via banner	via banner	1
BAUD_RATE		1	1	1	2
SEND_TIMEOUT		1	1	1	1
COMMAND_DELAY_SEC		2	2	2	
RESET_AT_COMMAND		1	1		2
RESET_DELAY_QCX216_SEC		1			2
RESET_DELAY_MDM9X07_SEC			1		2
RESET_DELAY_SDX35_SEC				unused	
STARTUP_MESSAGE_STRING		2	2	2	
LINE_DIVISOR_STRING		8	8	10	
SIM_CCID / IMEI / IMSI_LEN		1 each	1 each	1 each	1 each
BRANCH_NAME_EAGLE / NETPRISMA / QUECTEL		1 each	1 each	1 each	
AT_CMD_MIN_LINE_TO_PARSE		1	1	1	
PLATFORM_<module>		QCX216	MDM9X07	SDX35	
AT_CMD_RESPONSE_LINE					

main.py uses no constants. AT_CMD_RESPONSE_LINE and RESET_DELAY_SDX35_SEC are defined but not used anywhere.

2.6.1 Call flow with the shared files

Each step of a run, top to bottom, with the constants (amber) and test_case.py data (purple) it uses. The arrow on the left is the loop over the 44 test cases.



Figure 3 - Call flow from main.py into a module script

2.6.2 Data flow: from the inputs to expected responses

The rows in test_case.py start with placeholder values. Before the first AT command is sent, the module script overwrites 18 of them with values from the Excel sheet and the SIM input. TestCaseIndex turns a row name such as CD_01_003_5 into its position in the list (10).

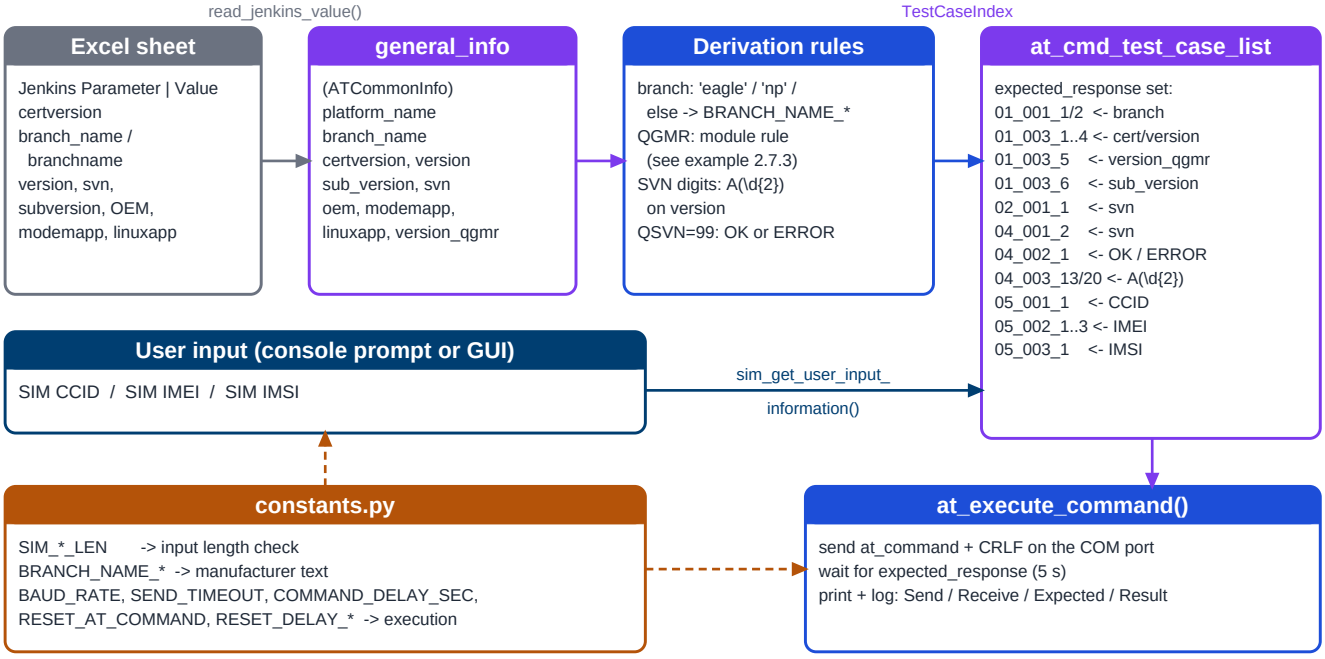


Figure 4 - Data flow from the Excel sheet and SIM input to the test list

2.7 Worked examples

These examples were produced by running the real backend functions on a copy of src/, with the sample sheet below and a simulated serial port. The values are the defaults in test_case.py, which match the existing build I4916QGLLGR01A02M04_ONT_A0.300.A0.300_V01.

2.7.1 Example input

Jenkins Parameter	Value	User input	Value
branch_name / branchname	R01_1075014_Eagle_73	SIM CCID	8960152241058850614F
certversion	I4916QGLLGR01A02M04	SIM IMEI	016580000000000
version	I4916QGLLGR01A02M04	SIM IMSI	502152107380061
subversion / svn	V01 / 02	COM port	COM5
OEM / oem_id	ONT	JIRA ID	QCX216-.. / SW9X07-.. / SDX35-..
modemapp / linuxapp	A0.300 / 1.100.100		

2.7.2 Expected responses after excel_extract_parameter()

Same input, three modules. Rows that differ between modules are highlighted.

TestCaseIndex	Command	QCX216	MDM9X07	SDX35
CD_01_001_1	AT+GMI	Eagle Wireless	Eagle Wireless	Eagle Wireless
CD_01_003_1	AT+GMR	I4916QGLLGR01A02M04	I4916QGLLGR01A02M04	I4916QGLLGR01A02M04
CD_01_003_3	ATI	<certversion>	<version>	<version>
CD_01_003_5	AT+QGMR	I4916QGLLGR01A02M04_ONT_A0.300.A0.300	I4916QGLLGR01A02M04_A0.300.1.100	I4916QGLLGR01A02M04_A0.300.1.100
CD_01_003_6	AT+CSUB	V01	V01	V01
CD_02_001_1	AT+EGMR=0,9	02	02	02
CD_04_002_1	AT+QSVN=99	OK	ERROR	OK (skipped)
CD_04_003_13	AT+EGMR=0,9	02	02	02 (skipped)
CD_05_001_1	AT+QCCID	8960152241058850614F	8960152241058850614F	8960152241058850614F
CD_05_002_1	AT+GSN	016580000000000	016580000000000	016580000000000
CD_05_003_1	AT+CIMI	502152107380061	502152107380061	502152107380061

In this sample certversion and version are equal, so ATI expects the same text in every module; the row shows which Excel field each module reads.

2.7.3 QGMR version derivation

```
QCX216  : version + '_' + OEM + '_' + modemapp + '.' + modemapp
          OEM 'ONT' is not inside version, modemapp present
          = I4916QGLLGR01A02M04 + _ONT + _A0.300 + .A0.300
          = I4916QGLLGR01A02M04_ONT_A0.300.A0.300   (matches the build name)

MDM9X07 : certversion + '_' + modemapp + '.' + first two parts of linuxapp
SDX35     linuxapp '1.100.100' -> '1.100'
          = I4916QGLLGR01A02M04_A0.300.1.100
          (modemapp or linuxapp 'None' -> certversion only)
```

2.7.4 Branch name and SVN rules

branch_name in Excel	Rule hit	AT+GMI / AT+CGMI expect
R01_1075014_Eagle_73	contains 'eagle'	Eagle Wireless
R02_NP_Release_5	contains 'np'	NetPrisma
R04_Main_2	no match	Quectel (default)
R05_Input_Fix	'np' inside 'iNPut'	NetPrisma - accidental match, see B9

```
re.search(r'A(\d{2})', 'I4916QGLLGR01A02M04')  ->  'A02'  ->  group(1) = '02'
-> CD_04_003_13 and CD_04_003_20 (AT+EGMR=0,9 after the 'FF' / 'Ff' tests) expect '02'
```

2.7.5 at_execute_command() traces

A. Exact match - AT+GMI (exact_match=True, response_line=1). Reply from the modem:

```
AT+GMI\r\nEagle Wireless\r\n\r\nOK\r\n
lines = ['AT+GMI', 'Eagle Wireless', 'OK']     # empty lines dropped
lines[1] == 'Eagle Wireless'  -> PASS
```

B. Exact match with prefix - ATI (response_line=3, prefix 'Revision: '). This is the console block that the GUI parses (section 4.3):

```
[9/44]

Test Case : Pretest_LTE_CD-01-003 [3]
Send      : ATI
Receive   : ATI
Eagle Wireless
EG915Q
Revision: I4916QGLLGR01A02M04

OK

Expected  : I4916QGLLGR01A02M04
Result    : Pretest_LTE_CD-01-003 [3] : PASS
--------------------------------------------------------------------------------
```

lines[3] = 'Revision: I4916QGLLGR01A02M04'; removeprefix('Revision: ') leaves the cert version, so the test passes.

C. Substring match - AT+QSVN? (expected '02', exact_match=False):

```
AT+QSVN?\r\n+QSVN: 02\r\n\r\nOK\r\n    ->  '02' in buffer  ->  PASS
```

D. FAIL - AT+QGMR when the firmware reply has no modem app part:

```
Receive   : AT+QGMR / I4916QGLLGR01A02M04_ONT / OK
Expected  : I4916QGLLGR01A02M04_ONT_A0.300.A0.300
expected text never appears -> waits the full SEND_TIMEOUT (5 s) -> FAIL
```

E. Short reply - ATI (confirmed finding B1):

```
Reply: ATI\r\nRevision: I4916QGLLGR01A02M04\r\n
lines = ['ATI', 'Revision: I4916QGLLGR01A02M04']   # len 2 passes the >= 2 check
lines[3]  -> IndexError: list index out of range   # the whole run stops
```

2.7.6 Modem reset sequence (CD-04-003, QCX216 / MDM9X07)

Step	Command	Expected	What the loop does after it
1	`AT+QSVN="13"`	`OK`	sleep COMMAND_DELAY_SEC (0.1 s)
2	`AT+CFUN=1,1`	`OK`	matches RESET_AT_COMMAND: close port, sleep RESET_DELAY_<M>_SEC (30 s), open, clear buffers
3	`AT+QSVN?`	`13`	SVN survived the reboot
4	`AT+EGMR=0,9`	`13`	Both commands report the new SVN

In the GUI worker, open() is replaced by ATReadySerial, which also waits up to 20 s for the modem to answer AT after the 30 s sleep. SDX35 has no reset handling; its CD-04 rows are skipped.

2.7.7 Console run through main.py

```
Please Enter Username (-@eaglewireless.com) : name@eaglewireless.com
Please Enter COM port (e.g. COM5)           : COM5
Please Enter SIM CCID (20 digits)           : 89601522410588506l4F
Please Enter SIM IMEI (15 digits)           : 016580000000000
Please Enter SIM IMSI (15 digits)           : 502152107380061
Please Enter JIRA ID (e.g. SW9X07-10)       : sw9x07-10      # upper-cased

MDM9X07 Pre-Test          -> mdm9x07.pretest_mdm9x07(...)
```

2.8 Backend observations

#	Severity	Finding	Location
B1	Medium	IndexError (reproduced, example 2.7.5 E): exact-match parsing only checks len(lines) >= 2, but ATI reads lines[3]. A short or truncated response raises, which is not caught inside the module (only SerialException is), so the whole run aborts instead of failing one test.	<module>.py at_execute_command
B2	Medium	Substring grading stops reading as soon as the expected text appears anywhere in the buffer, including the echoed command. Short expectations such as "2" or "OK" can pass on a partial or unrelated response.	at_execute_command
B3	Medium	SDX35 UNSUPPORTED returns True, so the backend's 'Pass' total in the log includes skipped commands. The GUI counts them separately, so the log summary and GUI/report summary disagree.	sdx35.py at_execute_command
B4	Low	~95 % duplicated code across three module scripts. A fix to grading or logging must be applied three times; a shared engine with per-module rule objects would remove this.	qcx216/mdm9x07/sdx35.py
B5	Low	Global mutable state: the scripts overwrite expected_response in the shared test list and ATCommonInfo. Safe only because each GUI run uses a fresh process.	test_case.py, all modules
B6	Low	Excel lookup uses the first *.xlsx returned by glob, which can be an Excel lock file (~$...). The GUI guards this; the console entry (main.py) does not.	excel_extract_parameter
B7	Low	read_jenkins_value() reopens the workbook for every parameter (about 10 opens per run) and returns the string "None" instead of None.	main.py and module copies
B8	Low	The modules' __main__ blocks call pretest_<module>() with no arguments (TypeError). @dataclass on an IntEnum has no effect. The SIM check validates length only, not characters.	module files, test_case.py
B9	Low	Branch detection is a plain substring test: any branch name containing 'np' (e.g. 'R05_Input_Fix') becomes NetPrisma, see example 2.7.4.	excel_extract_parameter
B10	Info	Unused items: AT_CMD_RESPONSE_LINE, RESET_DELAY_SDX35_SEC, and general_info in main.py.	constants.py, main.py

3. GUI Analysis

gui.py is a self-contained Tkinter application. It imports the backend as a library for read-only work (Excel parsing, test list, constants) and executes runs in a separate worker process. It also contains the JIRA client, the Windows credential store, the PDF report writer and the worker entry point.

3.1 Structure of gui.py

Lines	Section	Main items
69-123	Settings	Worker markers, login hash and salt, JIRA URLs and timeouts, spinner, credential target
133-223	Branding	resource_path(), load_logo_emblem() - connected-component cut of the eagle from the logo
225-335	Module profiles	ModuleProfile dataclass, MODULE_PROFILES (accent, entry, reset delay, Excel keys, notes), module_from_jira()
337-393	Worker	run_worker(): stdout setup, ATReadySerial patch, run, end markers
395-470	AT port probe	probe_at_port() with a lock, wait_for_at_ready() (AT repeated until OK)
472-531	Excel helpers	ExcelInfo, stored_excel_files(), detect_excel(), read_excel_values()
533-775	JIRA + credentials	jira_open/check_token/check_topic/reachable/download_excel/post_result, load_token/save_token (CredReadW/CredWriteW)
777-815	Parser	preview_expected(), RE_INDEX ... RE_TOTAL, TestRecord
816-1006	Theme + report	COLORS, fonts, px() DPI scaling, write_report_pdf()
1010-1222	Tooltip + login	Tooltip, login_password_hash() (PBKDF2-SHA256, 600k), LoginWindow
1227-3853	PretestApp	Layout, search, online mode, JIRA/token/port checks, run control, parser, results, clean-up
3860-3882	Entry	main_application(): worker branch or login, then main window

3.2 Start-up and login

- `main_application()` first checks `sys.argv[1] == '--worker'`; if so it becomes a worker and never shows a window.
- Otherwise it sets the AppUserModelID (taskbar icon), enables DPI awareness and opens `LoginWindow`.
- Login rule: the username must end with @eaglewireless.com (same rule as main.py); the password is compared as a PBKDF2-SHA256 hash with `hmac.compare_digest`. On success the username becomes the Reporter and the JIRA account for online mode.

3.3 Main window

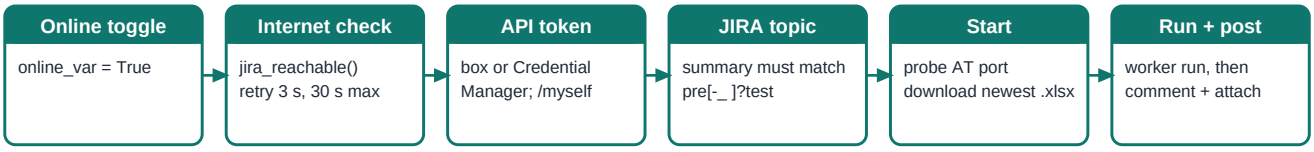
Area	Widgets	Behaviour
Header + module selector	Title, module badges, technology badge, Upload Excel, Reset View, Search	Accent colour follows the module selected from the JIRA ID; search appears after a run
JIRA Ticket card	JIRA ID, Online/Offline toggle, internet button, ? help, API token	Module derived live from the JIRA ID; online checks chained (see 3.5)
Connection & SIM card	COM port combobox + refresh, CCID / IMEI / IMSI with counters and tooltips	Port probed 400 ms after selection; DM/NMEA/GPS ports rejected by name
Tabs	Build Parameters, Module Profile	Excel values and derived QGMR preview; per-module notes
Pre-check + actions	Chips JIRA, AT port, Excel/API, CCID, IMEI, IMSI; Start, Stop	Start only enabled when every chip is OK
Right column	Progress card, Test Cases Treeview, detail pane, live log console	Updated from the worker output; reset countdown; download menu

3.4 Pre-check model

`_precheck()` returns a dict of `(state, reason)` with state ok / checking / bad. `_update_start_state()` paints the chips, enables Start, and writes the reasons into the status line. Asynchronous checks keep a `(value, status, reason)` tuple plus a check_id so that a stale result is ignored:

Chip	Source of truth	OK when
JIRA	module_from_jira(), topic_state	ID contains QCX216 / SW9X07 / SDX35; online: summary contains 'Pretest'
AT port	port_state via probe_at_port()	Selected port answered AT with OK
Excel / API	detect_excel() or token_state	Offline: one readable sheet, no lock file. Online: internet OK and token valid
CCID / IMEI / IMSI	_sim_ok()	Exact length; IMEI/IMSI digits only, CCID alphanumeric

3.5 Online mode (JIRA)



Each check runs in a daemon thread; the Tk loop polls a shared dict with after(100 ms). A check_id counter discards stale results when the user edits the JIRA ID / token or switches mode. Failures show the '?' help box and block Start via the pre-check chips.

Figure 5 - Online mode check chain

- Authentication: HTTP Basic with login username + Atlassian API token, built by `jira_open()` on urllib.
- A valid token is saved in Windows Credential Manager (target 'EaglePretest JIRA API Token (user)') and auto-filled next time; copy/cut from the token box is blocked.
- At Start, previous sheets are deleted, the newest .xlsx attachment is downloaded to a temp folder, validated, and copied to the application folder where the backend expects it.
- After a completed (not stopped) run, a PDF report and the log are attached to the ticket and a wiki-markup comment links them.

3.6 Run control and output parsing

- `start_test()`: validate, probe the port in a thread, then `_download_and_launch()` (online) or `_launch_test()`.
- `_launch_test()`: selects the module, records run info for the report, deletes old logs, and starts the worker with `subprocess.Popen` (CREATE_NO_WINDOW, cwd = app folder, UTF-8, stdout+stderr piped).
- `_reader()` thread pushes each stdout line into a queue; `_drain_output()` handles up to 500 lines every 50 ms on the Tk thread.
- `_handle_line()` is a small state machine (current record, is_in_receive) that maps lines to table rows - see section 4.3.
- Stop: `process.terminate()` after confirmation; the partial log is kept and found by `_find_run_log()` when no marker was printed.

3.7 Results and housekeeping

- Download Result menu: log copy (.txt) or `write_report_pdf()` (landscape A4: run info, JIRA link, all 44 rows with the actual response). Both are named EaglePretest_Report_YYYYMMDD_HHMMSS.
- New session: Excel sheets and logs from the previous session are deleted at start-up.
- `_tick()` (500 ms): elapsed time, reset countdown, and reload when the .xlsx in the folder changes.

3.8 Threading model

All widget updates run on the Tk main thread. Blocking work (serial probe, HTTP calls, stdout reading) runs in daemon threads that only write into a dict or queue; the main thread polls them with `after()`. A module-level `probe_lock` ensures the COM port is never opened twice by concurrent probes. This avoids Tk cross-thread calls.

3.9 GUI observations

#	Severity	Finding	Location
G1	High	The team password is printed in plain text in README.md, which defeats the PBKDF2 hash stored in gui.py. The login is also a client-side check only: anyone with the source or exe can bypass it.	README.md, LOGIN_PASSWORD_HASH
G2	Medium	gui.py is 3,882 lines with PretestApp at about 2,600 lines. JIRA client, credential store, report writer and worker could move to separate modules to ease testing and maintenance.	gui.py
G3	Medium	Several backend rules are copied instead of shared: _sdx35_unsupported(), the username rule, the JIRA-to-module map, the SIM placeholders. A backend change can silently diverge.	gui.py 86, 265, 1210, 2883
G4	Low	_tick() uses next(glob('*.xlsx')) which includes ~$ lock files, while stored_excel_files() excludes them; while a sheet is open in Excel the view may reset repeatedly.	gui.py 3469
G5	Low	preview_expected() runs the backend's excel_extract_parameter() in the GUI process, which mutates the GUI's copy of the test list (harmless today, because the table later shows the worker's values).	gui.py 786
G6	Low	Hard-coded strings: 'Anas Akasyah' in the header and footer, login date '21-10-2026', default module_var 'SDX35' (unused).	gui.py 1158, 1274, 2461
G7	Info	Polling with after(100) is used for every background task; a single generic helper would remove about ten near-identical _wait_* methods.	PretestApp

4. Connection between GUI and Backend

The GUI was written to use the backend 'as a service without modifying it'. It connects to the backend in two ways: as a **library** in its own process for read-only work, and as a **child process** whose console output is the integration interface for the actual test run.

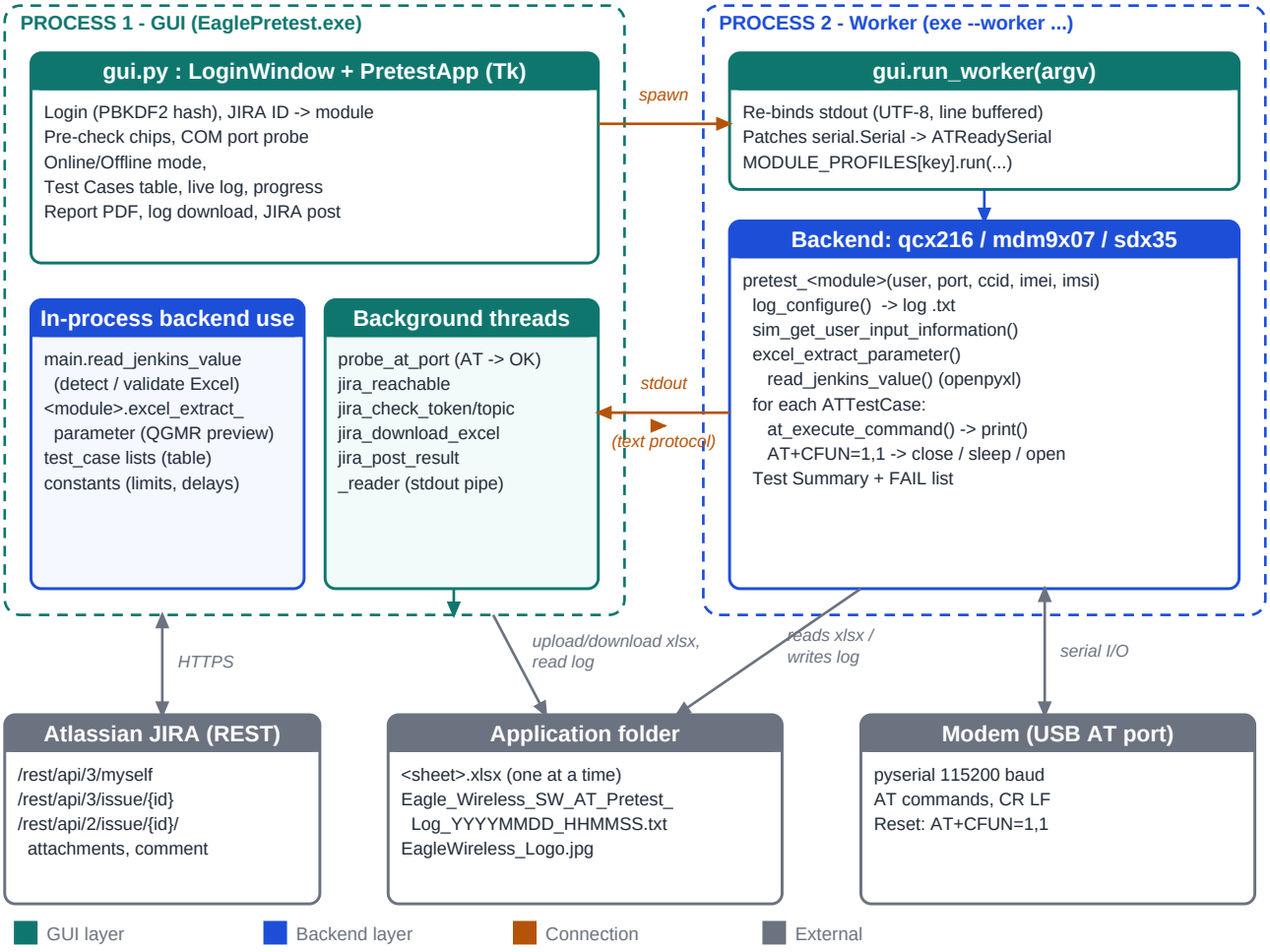


Figure 6 - Runtime architecture: two processes, one executable

4.1 Integration points

GUI side	Backend item	Mode	Purpose
`import constants as const`	`constants.py`	In-process	Version, baud rate, SIM lengths, reset delays, reset command
`detect_excel(), read_excel_values(), _store_excel()`	`main.read_jenkins_value`	In-process	Validate and display the Jenkins sheet
`preview_expected()`	`<module>.excel_extract_parameter, at_cmd_general_info_list`	In-process	Show the derived QGMR version before running
`_populate_table(), _render_profile(), _on_result()`	`at_cmd_test_case_list, TestCaseIndex`	In-process	Table rows, counts of skipped commands and resets, next command
`MODULE_PROFILES[key].run()`	`pretest_qcx216 / _mdm9x07 / _sdx35`	Worker process	Execute the run (getattr(service, entry))
`ATReadySerial (patch)`	`serial.Serial used inside the module`	Worker process	Wait for AT OK on every open, including after AT+CFUN=1,1
`_handle_line()`	`print() output of at_execute_command / summary`	Pipe	Live table, progress, status
`_launch_test cwd + app_directory()`	`Path(sys.executable).parent`	File system	Both sides find the .xlsx and the log in the same folder

4.2 Process launch contract

```
frozen : EaglePretest.exe --worker <KEY> <username> <COMx> <ccid> <imei> <imsi>
source : python -u gui.py --worker <KEY> <username> <COMx> <ccid> <imei> <imsi>
env    : PYTHONUNBUFFERED=1, PYTHONIOENCODING=utf-8     cwd: application folder
KEY    : QCX216 | MDM9X07 | SDX35   (from module_from_jira(JIRA ID))
```

Running each test in a fresh process gives three properties the backend cannot provide on its own: fresh global state (the test list and ATCommonInfo are mutated during a run), a fresh logging.basicConfig (which only takes effect once per process), and a safe Stop that releases the COM port by terminating the process.

4.3 Stdout protocol

The backend's human-readable console output doubles as a machine protocol. Each `at_execute_command()` call prints one block; the GUI matches it with anchored regular expressions:

Backend line	GUI regex / check	GUI action
`[12/44]`	`RE_INDEX ^\[(\d+)/(\d+)\]$`	Select record 12, set progress maximum to 44
`Test Case : Pretest_LTE_CD-01-003 [1]`	`RE_TEST_CASE`	(informational)
`Send : AT+GMR`	`RE_SEND`	record.send
`Receive : <first line>`	`RE_RECEIVE`	Start collecting multi-line response
`(further response lines)`	`is_in_receive state`	Appended to record.receive
`Expected : <value>`	`RE_EXPECTED`	End of response; record.expected
`Result : ... : PASS|FAIL|UNSUPPORTED`	`RE_RESULT`	_on_result(): row colour, counts, progress, reset countdown
`Test Summary`	`startswith`	is_summary_seen = True (run completed)
`Cannot open... / Error... / Invalid SIM...`	`startswith / contains`	Kept as failure reason
`[Port] COMx ready after n s`	`(worker only)`	Shown in console
`@@GUI_LOG_FILE@@ <path>`	`MARKER_LOG_FILE`	Enable Download, remember log path
`@@GUI_WORKER_DONE@@`	`MARKER_EXIT`	Ignored; EOF on the pipe ends the run

The exact column alignment matters: 'Send' is followed by six spaces and 'Receive' by three. Any change to the backend's print format (spacing, labels, the [i/N] line) silently breaks the live table even though the run and log still work. tests/test_parser.py with fake_worker.py covers this contract on the GUI side, but there is no test on the backend side that pins the format.

4.4 End-to-end sequence of a run

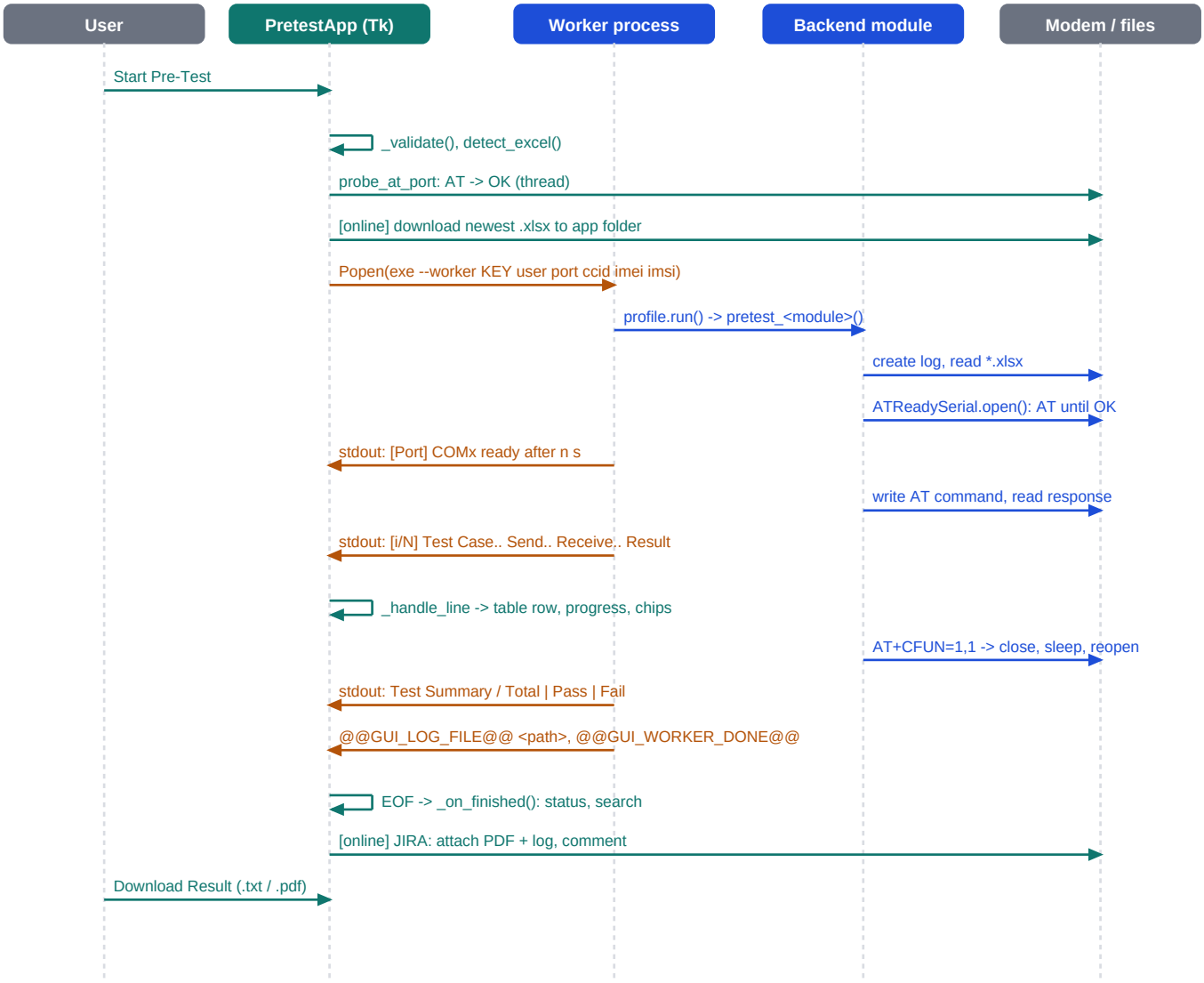


Figure 7 - Sequence of one pre-test run (online steps marked)

4.5 Shared file-system contract

Artifact	Written by	Read by	Rule
`<sheet>.xlsx in app folder`	GUI (upload / JIRA download)	Backend (first *.xlsx)	GUI keeps exactly one sheet; deletes old ones; refuses lock files
`Eagle_Wireless_SW_AT_Pretest_Log_*.txt`	Backend (logging)	GUI (download, JIRA attach)	Path reported by marker; old logs deleted at session start and before each run
`EaglePretest_Report_*.pdf / .txt`	GUI	User / JIRA	Same timestamp as the run start
`EagleWireless_Logo.jpg`	Build (bundled)	GUI	Found next to the exe or in _MEIPASS

4.6 Rules defined twice

Rule	Backend	GUI copy	Risk if changed in one place
JIRA ID -> module	`main.py main_application`	`JIRA_PROJECTS, module_from_jira`	Wrong module run
Username domain	`main.py`	`LoginWindow.login`	Inconsistent login
SDX35 skip list	`sdx35.at_execute_command`	`_sdx35_unsupported`	Table shows N/A for a run row or vice versa
SIM-based expected rows	`sim_get_user_input_information`	`_expected_placeholder`	Wrong placeholder text
Reset delay per module	`constants + module loop`	`ModuleProfile.reset_delay`	Countdown does not match the real wait
Excel keys per module	`excel_extract_parameter`	`ModuleProfile.excel_keys`	Build Parameters tab shows wrong fields

4.7 Assessment of the coupling

- **Strengths:** the backend stays usable on its own; process isolation solves global state, logging and safe stop; the GUI never blocks on serial or network I/O; the patch of serial.Serial adds a readiness wait without touching module code.
- **Weaknesses:** the interface is an informal text format; module rules are mirrored in MODULE_PROFILES; the GUI depends on internal backend functions (excel_extract_parameter, TestCaseIndex) rather than a stable API; result counting differs between the two sides (B3).
- **Suggested direction:** have the backend emit one structured line per result (for example a JSON line with a prefix such as @@RESULT@@) next to the human text, and expose the skip / reset / Excel-key rules from each module so the GUI reads them instead of copying them.

5. Prioritised Recommendations

Priority	Action	Ref.
1	Remove the plain-text team password from README.md and rotate it; share it through a secure channel.	G1
2	Guard the exact-match index (len(lines) > response_line) and catch per-command exceptions so one bad response fails one test instead of ending the run.	B1
3	Count UNSUPPORTED separately in the backend summary so the log matches the GUI and the report.	B3
4	Read the full response (until OK/ERROR or a quiet period) before grading, and match expected values on response lines rather than anywhere in the buffer.	B2
5	Add a structured result line to the stdout protocol and a backend test that pins the format.	4.3
6	Merge the three module scripts into one engine with per-module rule objects, and let MODULE_PROFILES import those rules.	B4, G3
7	Split gui.py into packages (jira, credentials, report, worker, views).	G2
8	Clean-ups: lock-file filter in _tick(), workbook opened once, whole-word branch matching, remove hard-coded names/dates and broken __main__ blocks.	B6-B10, G4-G6

Appendix A - Test suite

File	Covers
`test_login.py`	Username rule, hashed password, version from constants
`test_internet.py`	Online/Offline toggle, internet check, ? help box
`test_token.py`	Token check, remembered token, copy blocked
`test_topic.py`	Pretest JIRA check, JIRA not found
`test_download.py`	Newest sheet download, old sheets removed, sheet open in Excel
`test_result.py`	PDF report, JIRA comment and attachments
`test_offline.py`	Upload, input checks, session clean-up, full offline run
`test_parser.py`	Worker output to table: PASS/FAIL/SKIP, responses, progress, stop, reset, crash
`support.py / fake_worker.py`	Mock JIRA on 127.0.0.1, in-memory token store, fake worker output

All tests target the GUI. The backend grading logic (at_execute_command, QGMR derivation, Excel extraction per module) has no unit tests; a fake serial port would allow them without a modem.